

Alpa and PartIR

Aaron Bartlett, Brandon Chao, Uygur Tepe
(aaronbar, bchao, tepe)

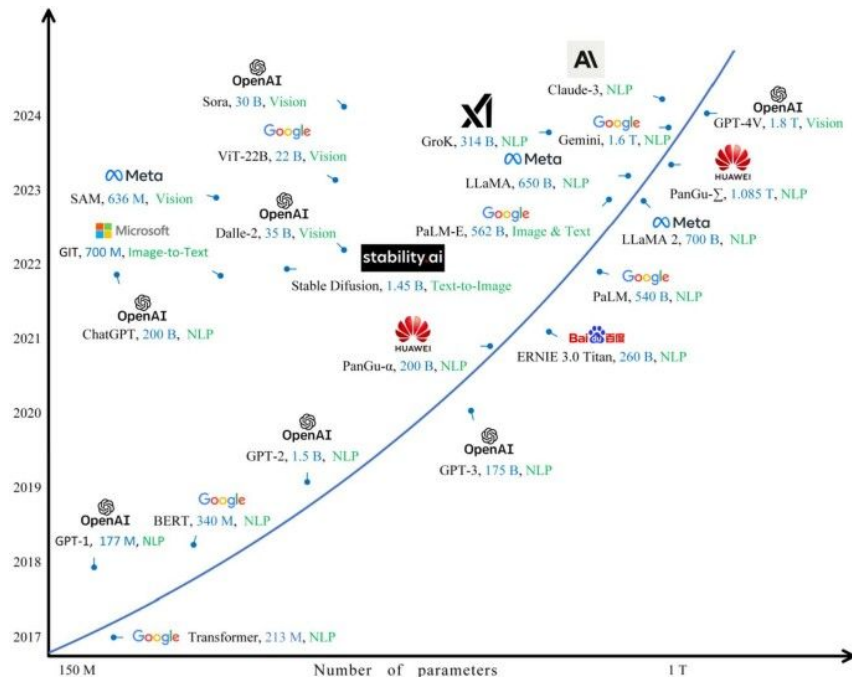
Sept 16th, 2025

Background

What is the Problem

LLM Model size has been increasing ~10x every year

Training Large Models: Not just an ML Challenge but also a Systems Challenge



An overview of large AI models and their applications

System Challenges

What if input dataset is very large?

Easy: Use data parallelism*, partition input data and replicate model

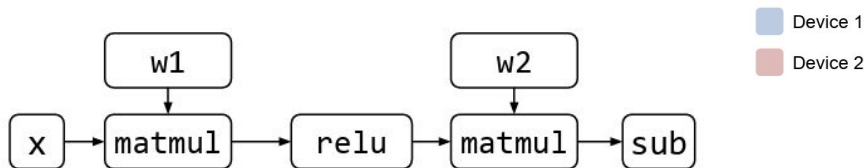
What if model is very large?

Hard: Need dozens of GPUs to hold the weight params and partition the model across them



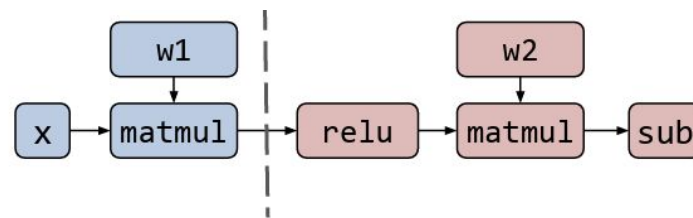
But how do we partition
the computational graph?

Partitioning Computational Graphs

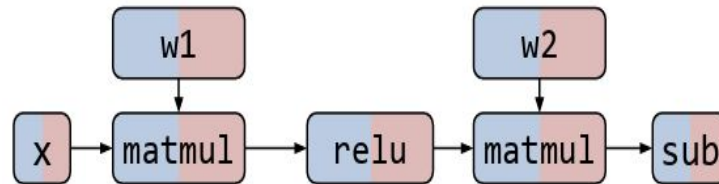


	Inter-op Parallelism	Intra-op Parallelism
Communication	Less	More
Device Idle Time	More	Less

Inter-operator (Pipeline) Parallelism:



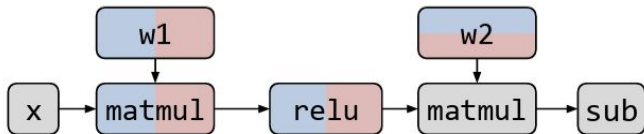
Intra-operator (Tensor) Parallelism:



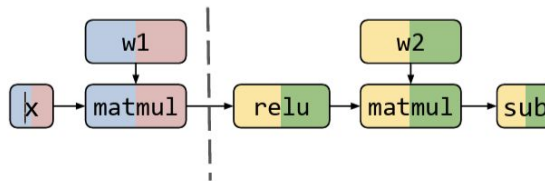
Partitioning Computational Graphs

■ Device 1
 ■ Device 2
 ■ Replicated
 ■ Row-partitioned
 ■ Column-partitioned

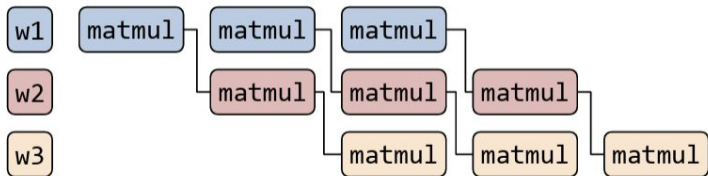
Multiple Intra-op Strategies for a single node:



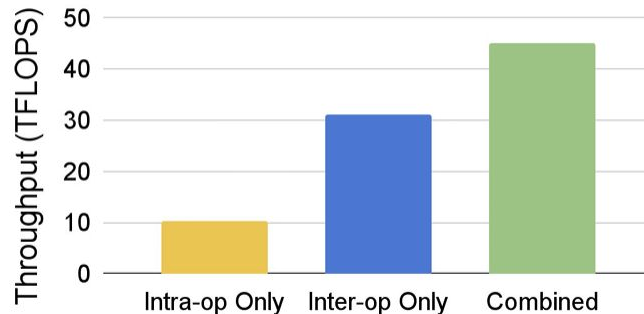
Combining Inter-op and Intra-op:



Execution pipelining for Inter-op parallelism:



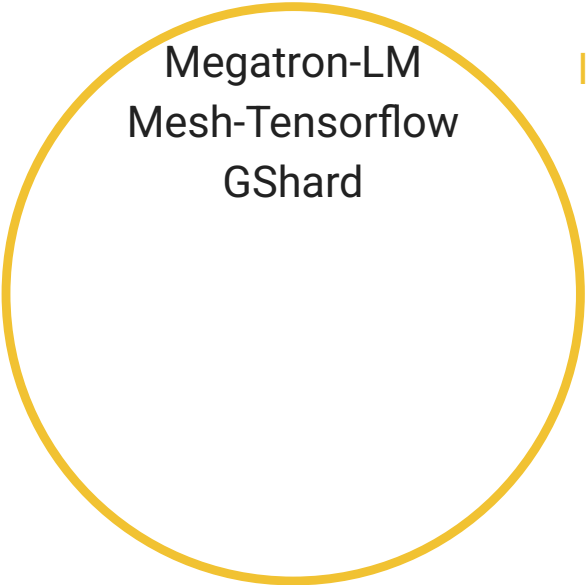
Training Throughput of an MoE Model



Prior Works - Intra-op Parallelism

- Megatron-LM (2019): Tensor parallelism - splits large operators across many GPUs
 - **Pros:** Enables scaling to billions of params on limited GPU memory
 - **Cons:** Required manual engineering of sharding strategies; heavy communication overhead between GPUs
- Mesh-TensorFlow/GShard (2018/2020): Provides higher-level API to partition tensors across a logical device mesh
 - **Pros:** More flexible than Megatron; supports different partitioning strategies (row, column, batch)
 - **Cons:** Also requires manual sharding annotations in model code; poor portability across hardware

Prior Works

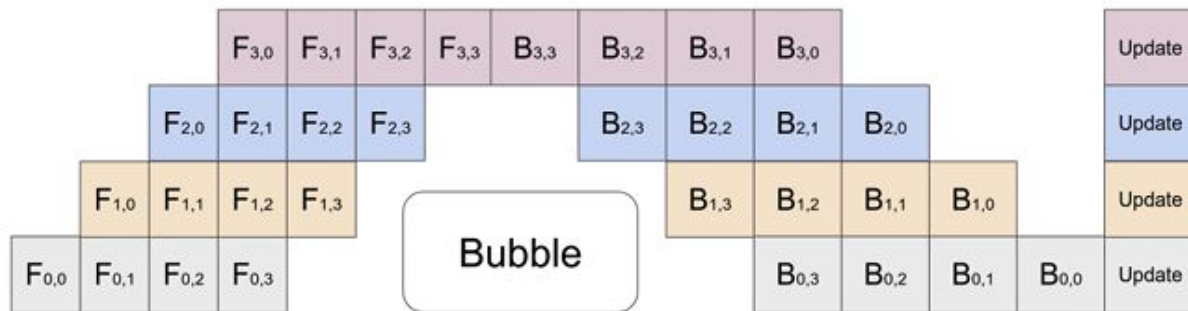


Megatron-LM
Mesh-Tensorflow
GShard

Intra-op Parallelism
(w/ operator-level)

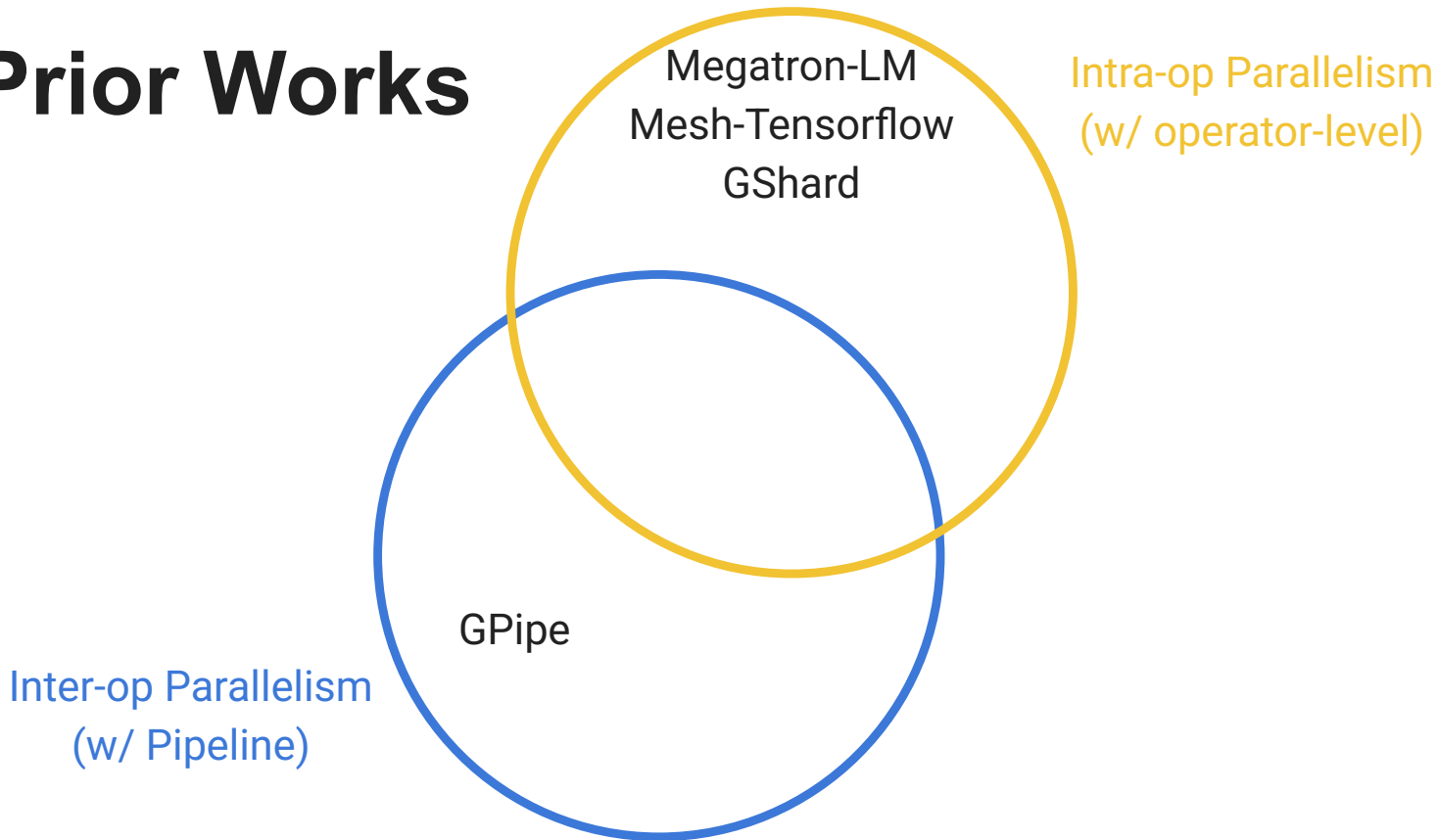
Prior Works - Inter-op Parallelism

- GPipe (2019): Split layers of a model across devices; feed micro-batches in a pipeline
 - **Pros:** Simple implementation, reduces memory footprint
 - **Cons:** “Pipeline bubbles” - some devices idle while waiting for others





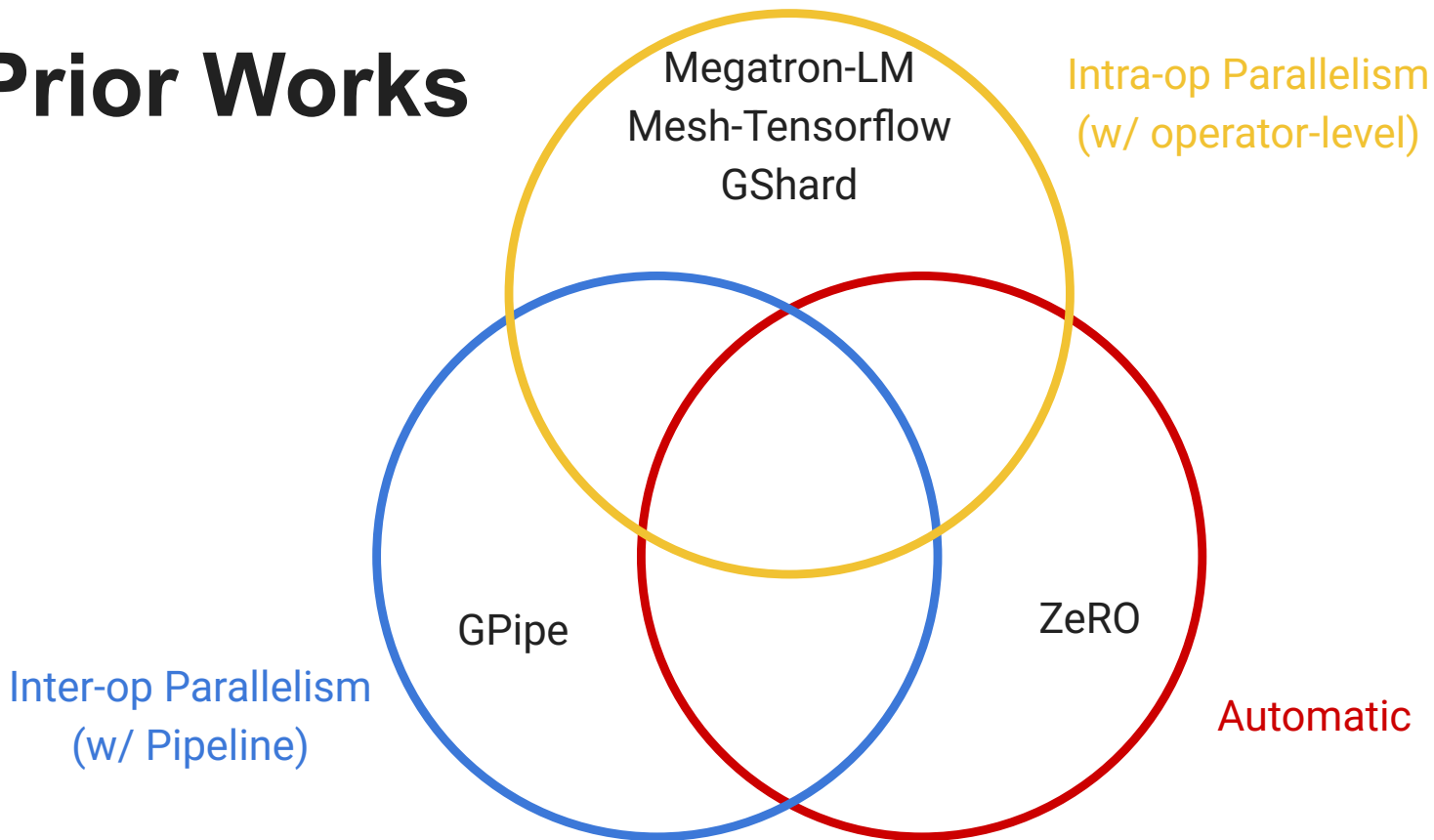
Prior Works



Prior Works - Automatic Systems

- ZeRO (2020): Automatically shards optimizer states, gradients and parameters across devices
 - **Pros:** Big memory savings; supports training trillion-parameter models
 - **Cons:** Focuses only on memory optimization; doesn't decide operator/pipeline parallelism

Prior Works



Prior Works - Combined Approaches

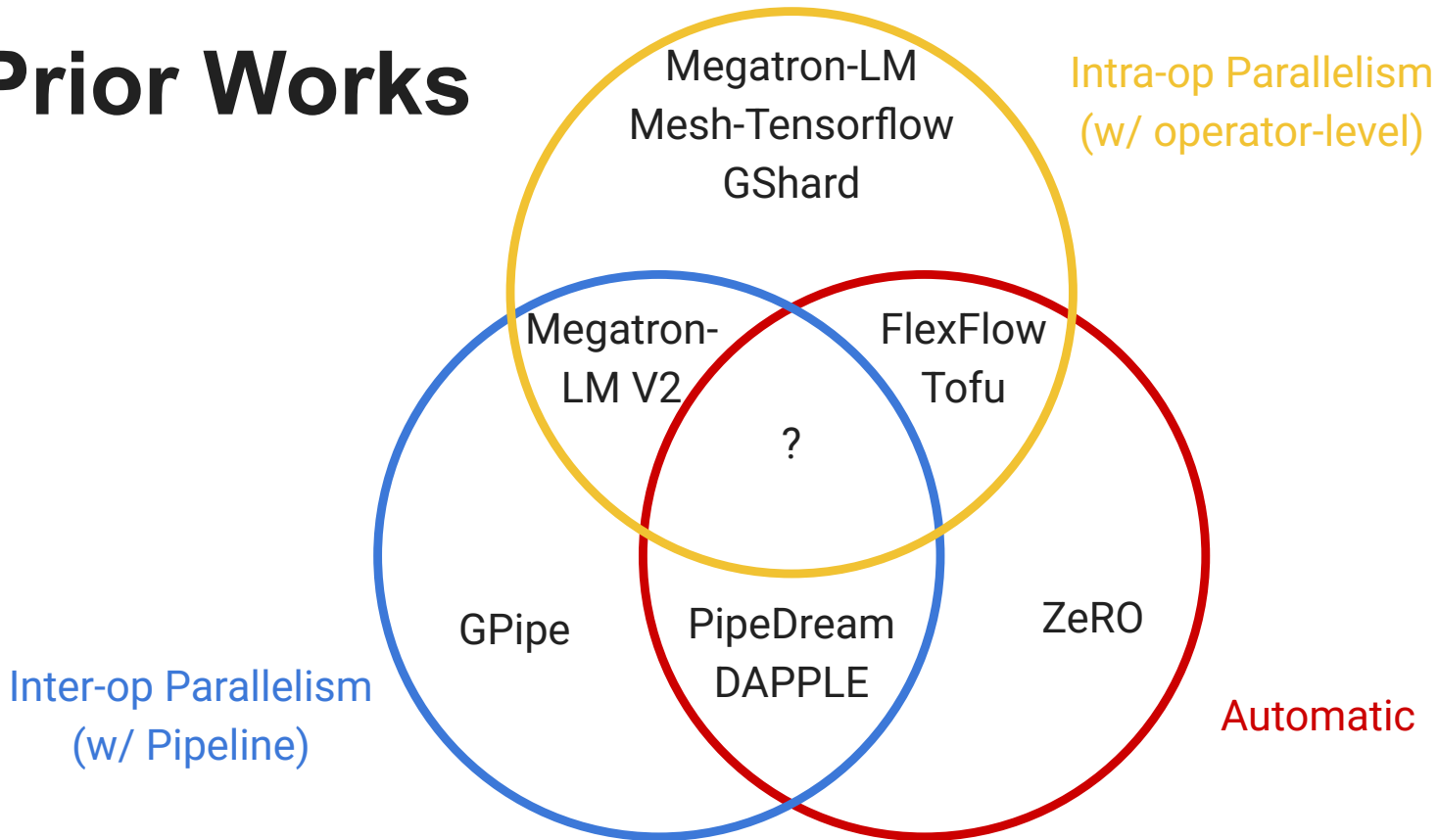
- Megatron-LM v2 (2021): Combines tensor parallelism (intra-op) with pipeline parallelism (inter-op)
 - **Pros:** Enables training of models with hundreds of billions of parameters
 - **Cons:** Requires manual configuration of both tensor and pipeline dimensions; not automated
- PipeDream/DAPPLE (2019/2020): Blend pipeline parallelism with automation to reduce bubbles and improve utilization
 - **Pros:** Pipeline execution without excessive idle time
 - **Cons:** Trade-off between weight staleness vs. throughput; limited flexibility for new models

Combined Approaches (cont.)

- Tofu/FlexFlow (2019/2018): Try to automatically search across intra-op combinations
 - **Pros:** Can discover non-obvious strategies that outperform manual ones
 - **Cons:** Very expensive search; impractical for today's massive models



Prior Works



Why is this Problem Important?

Model designs look almost the same as 5 years ago;
only the data and machines got larger, so scaling
them requires system innovation

**Automated parallelization would allow ML
developers to focus on models, not system
complexity**

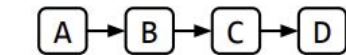
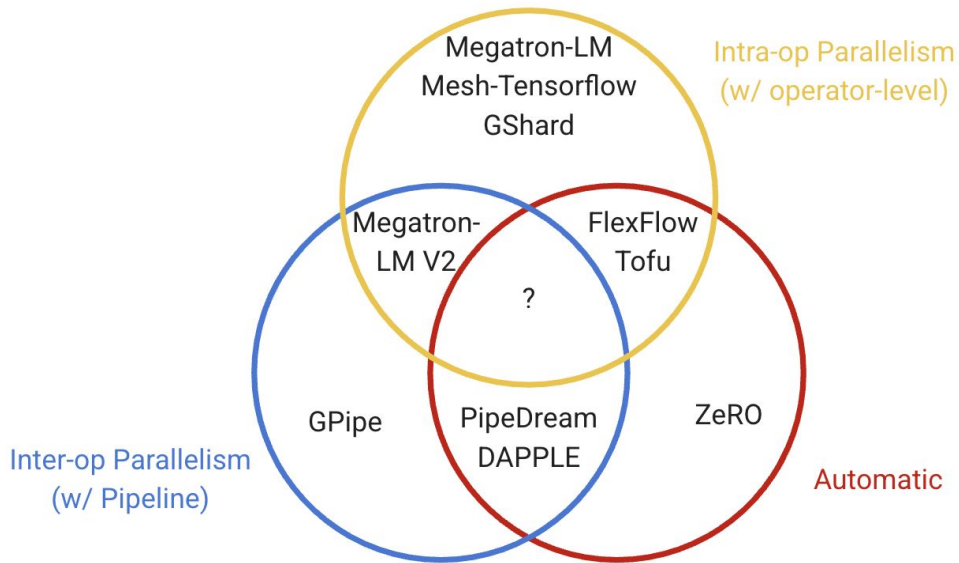
Alpa

**Automating Inter- and Intra-Operator Parallelism
for Distributed Deep Learning**

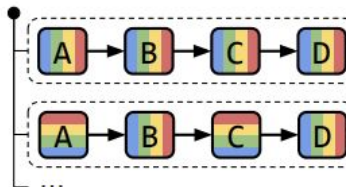
Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning

- Automates Compilation of a Given Graph
- Optimizes for both Inter- and Intra-Op Parallelism
- Applicable to most Large Deep Learning Models

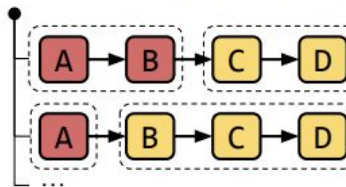
Building on Prior Works



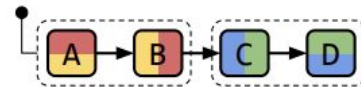
(a) Computational graph



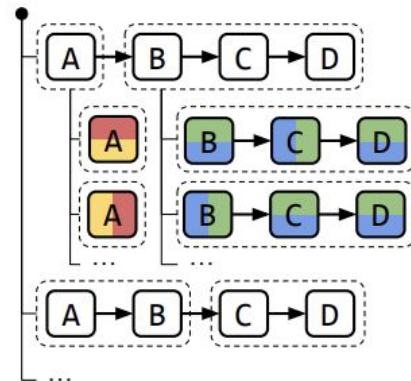
(c) The space of intra-operator parallelism (e.g., Tofu [55])



(d) The space of inter-operator parallelism (e.g., DAPPLE [17])



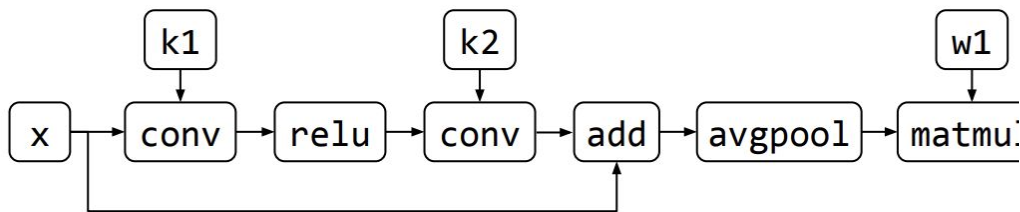
(b) Manual plan (e.g., Megatron-LM [40])



(e) Our hierarchical space

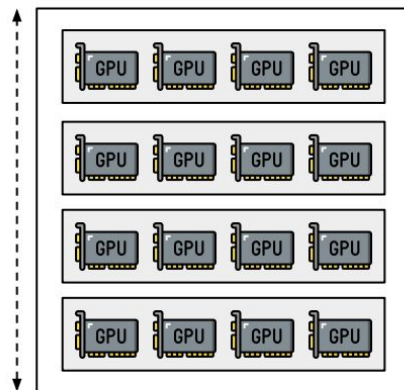
Alpa Inputs

1. Computational Graph
Operator Execution Plan and Dependencies
2. 2D Device Mesh
How many Nodes of how many GPUs



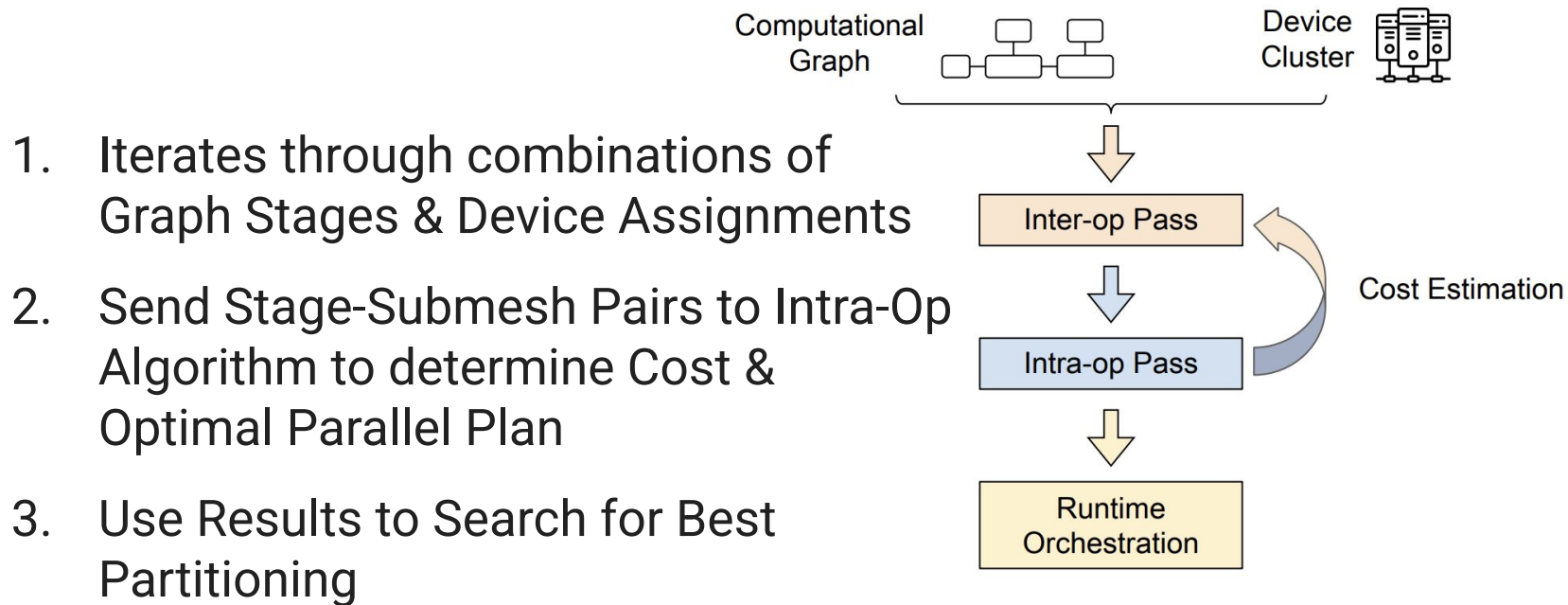
Nodes 🐢

Cluster (2D Device Mesh)

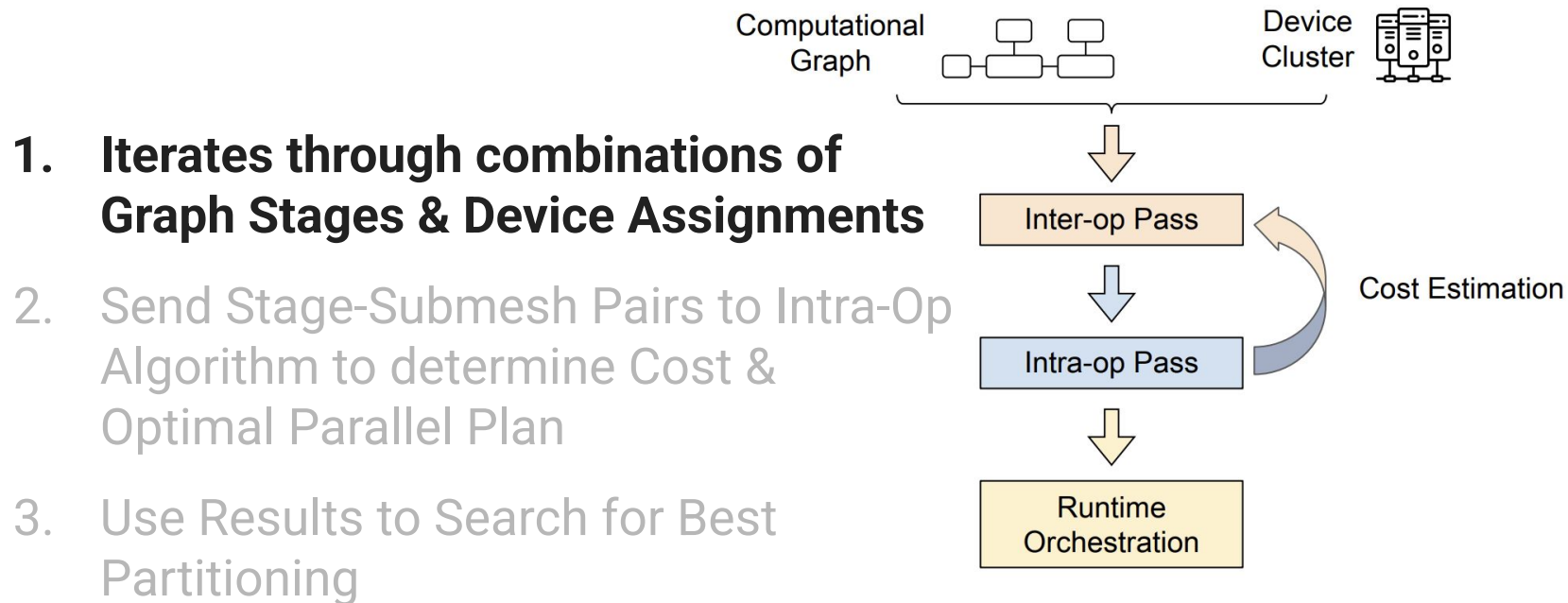


GPUs within a Node 🐭

Hierarchical Search Overview

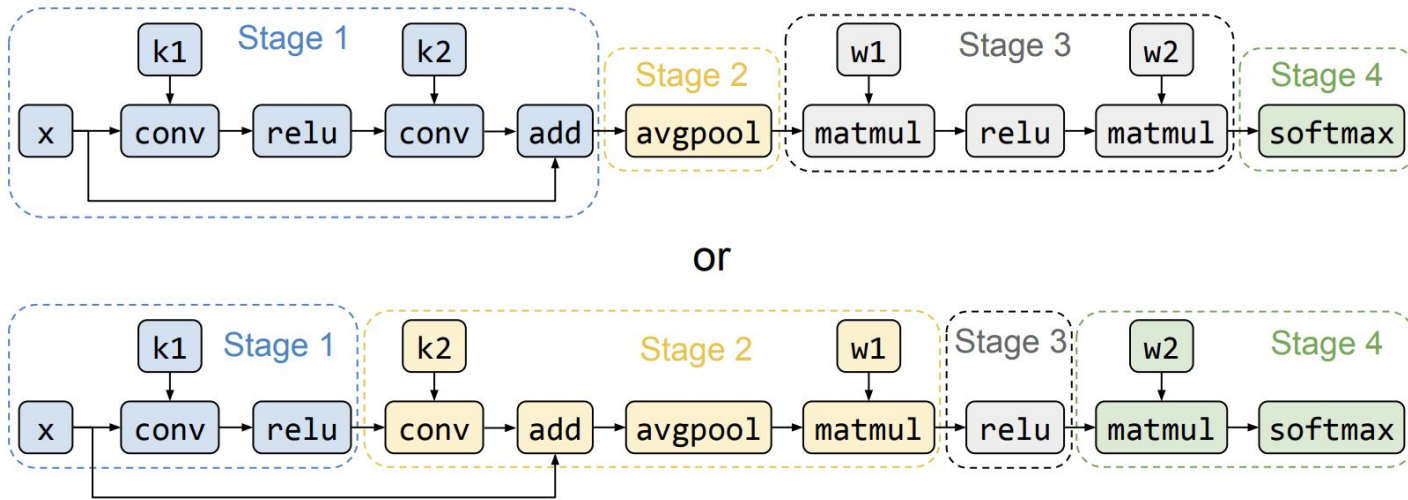


Hierarchical Search Overview

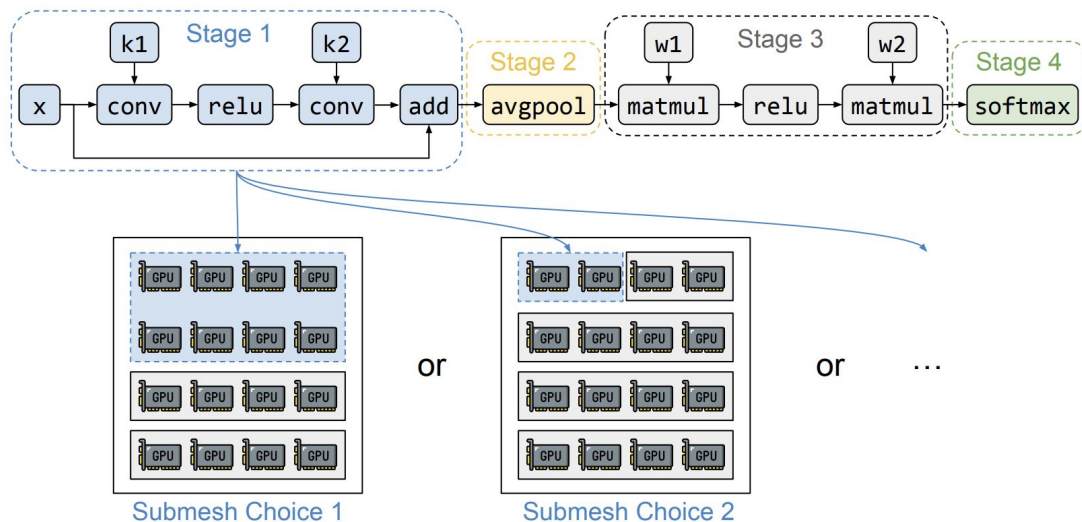




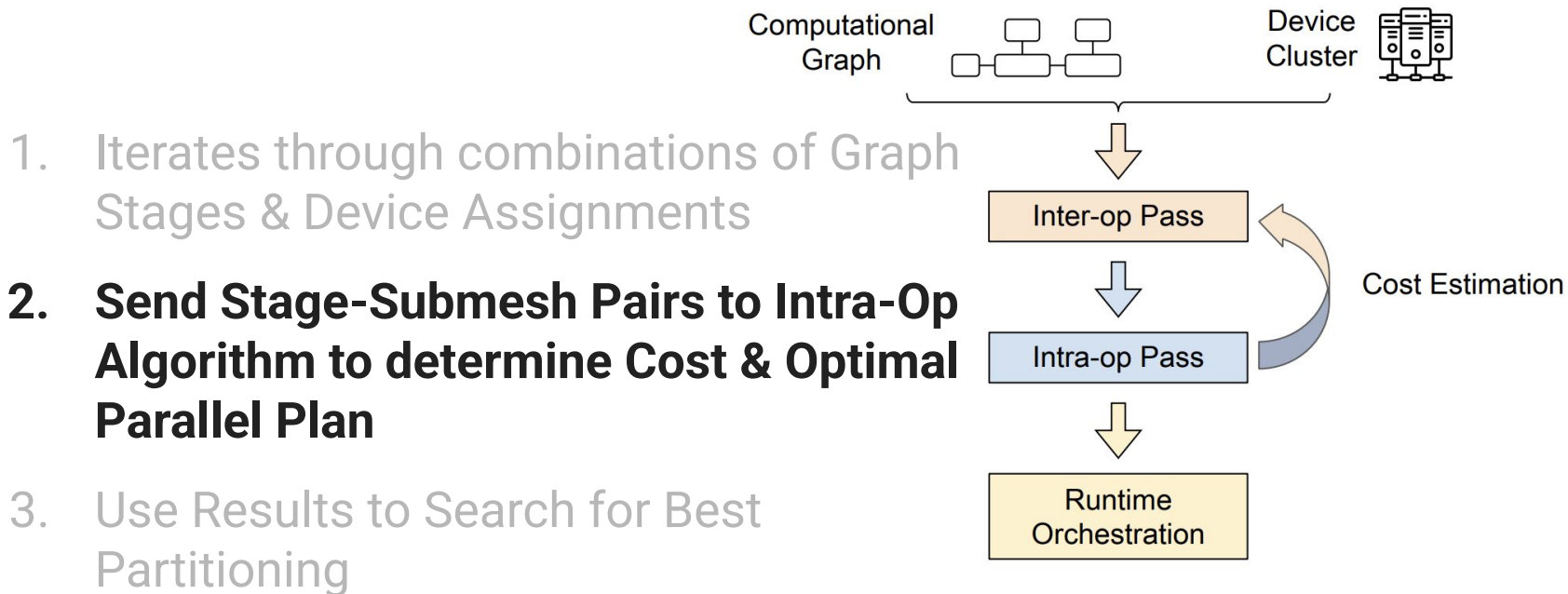
Inter-Op: Possible Graph Partitions



Inter-Op: Submesh Assignment

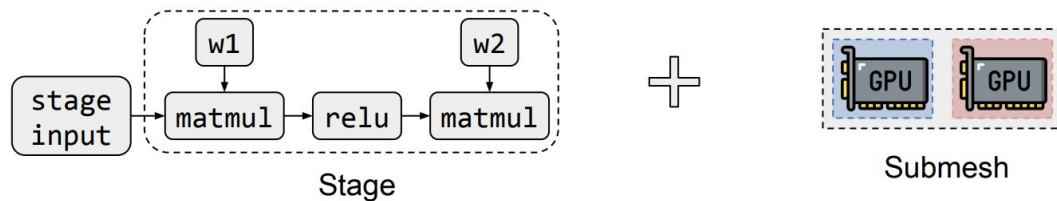


Hierarchical Search Overview



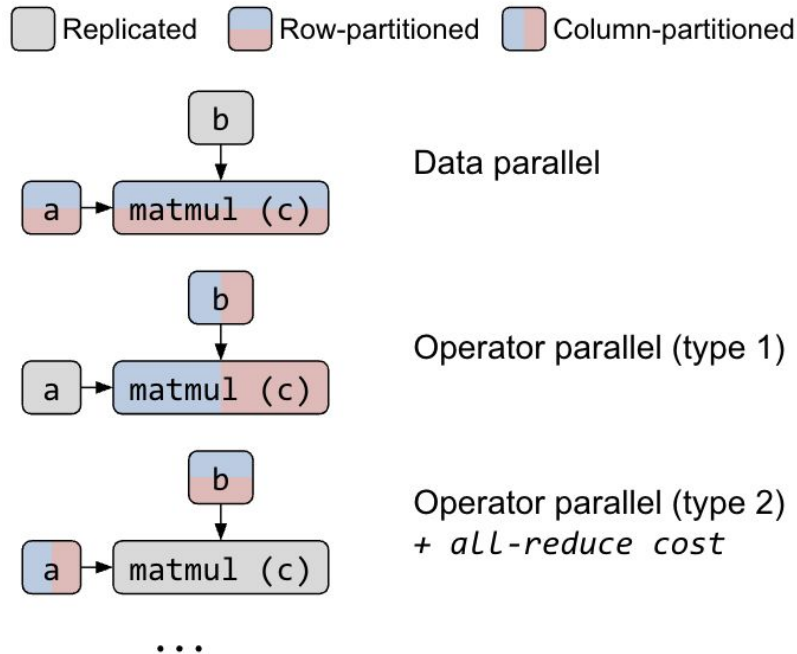
Intra-Op Pass: Inputs

1. Stage Computation Graph
2. Device Assignment



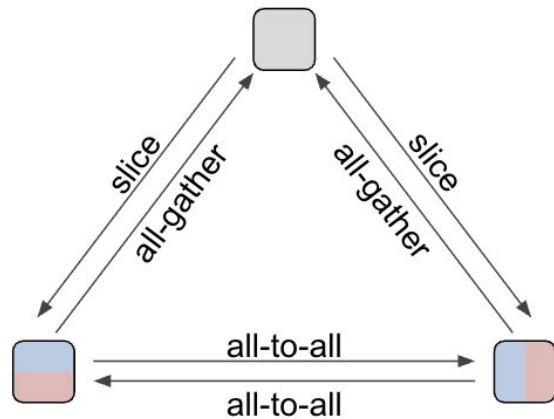
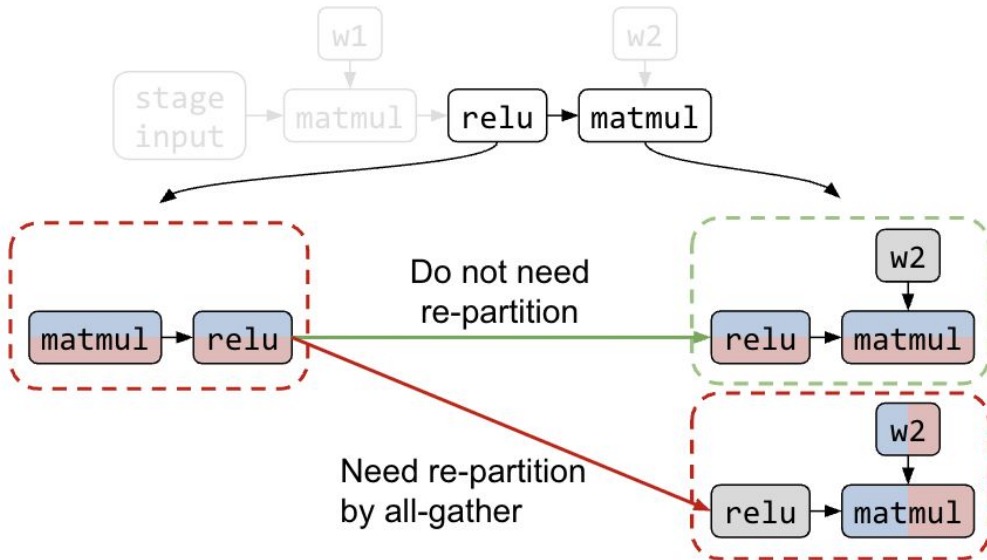
Intra-Op Pass: Parallel Schemes

Matrix Multiplication Example:



Intra-Op Pass: Repartition Cost

Replicated
 Row-partitioned
 Column-partitioned

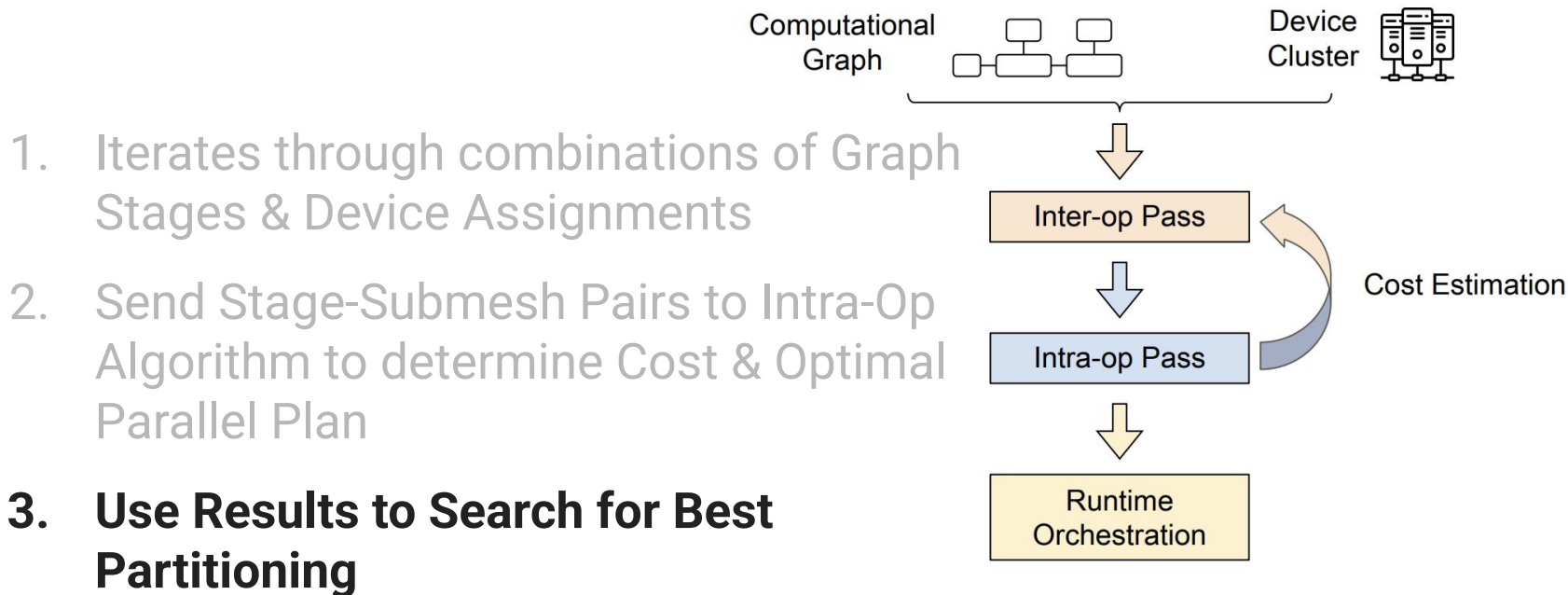


Intra-Op: Cost Calculation

Integer Linear Programming Problem

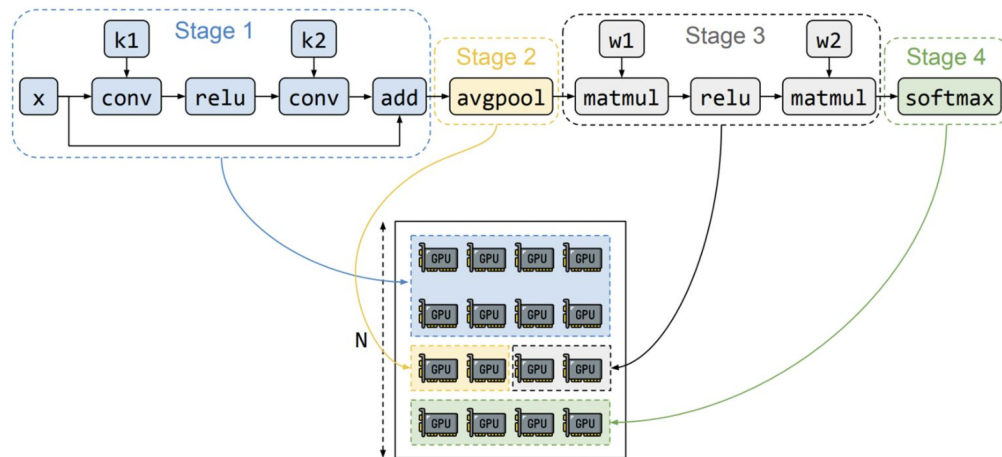
$$\min_s \quad \text{COMMUNICATION PLUS COMPUTATION COST} + \sum_{(v,u) \in E} \text{RESHARDING COST}$$

Hierarchical Search Overview



Inter-Op: Search Algorithm

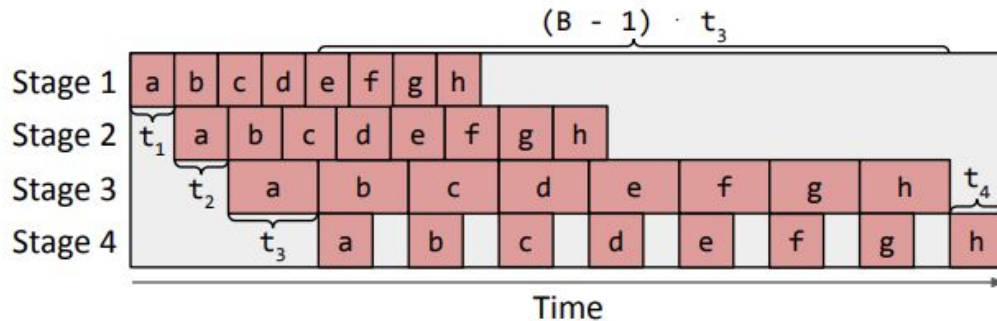
Assigns Stages and Submeshes using a Dynamic Programming Algorithm built around the 2D-Knapsack Problem



Inter-Op: DP Algorithm

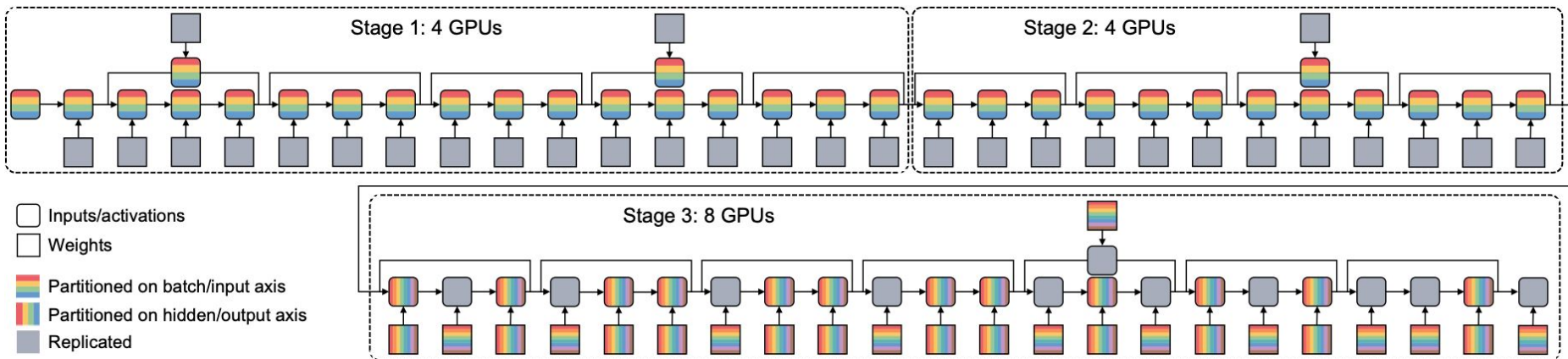
Optimizes for Total Latency:

$$T^* = \text{SUM OF ALL STAGES} + (B - 1) \cdot \text{LATENCY OF LONGEST STAGE}$$



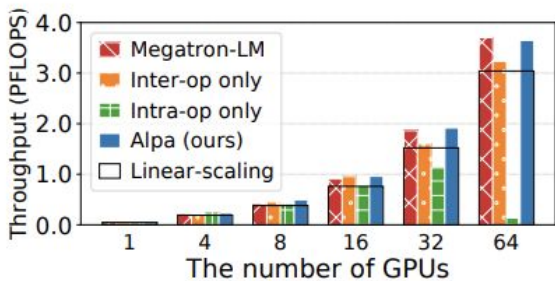
Alpa Results

Wide-ResNet Partition Results on 16 GPUs

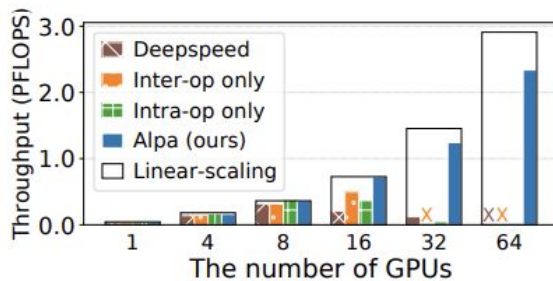


Alpa Results: Performance

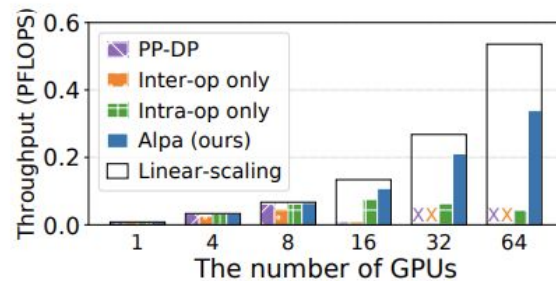
- Matches Performance of **Megatron-LM** for GPT-3, a model it was optimized for specifically
- Vastly Outperforms **DeepSpeed** on Mixture of Experts Models
- **No Viable System** existed for Wide-ResNet previously



(a) GPT



(b) MoE



(c) Wide-ResNet

Recap

- **Automates compilation** taking advantage of pipeline, tensor, and data parallelism
- **Matches or exceeds** performance of manually-designed architectures
- **“Black-Box Compilation”** - only user interaction is including the line `@alpa.parallelize`

Discussion

Pros

- Unified automation of **multiple parallelism types**
- Two-level hierarchical plan search/compile

Cons

- Static **linear pipeline schedule**
- Assumes **static computational graphs** and known tensor shapes

PartIR

Composing SPMD Partitioning Strategies for Machine Learning

Motivation

- Lots of parallelization techniques
- User written strategies are error prone
- Tightly integrated with ML code
 - Maintainability is bad when transitioning from different hardware specs

Proposed Solution

- Implement a methodology to specify different types of parallelism - agnostic to hardware specification
- Decouple ML implementation from partitioning problem

Proposed Solution

- Enable *Batch Parallelism* (BP), *Model Parallelism* (MP), and *Optimizer Sharding*
- Users can manually or automatically schedule these strategies



PyTorch APIs

```
from torch.distributed.tensor.parallel import ColwiseParallel, RowwiseParallel, parallelize_module

layer_tp_plan = {
    # by default ColwiseParallel input layouts is replicated
    # and RowwiseParallel output layouts is replicated
    "feed_foward.w1": ColwiseParallel(),
    "feed_forward.w2": RowwiseParallel(),
    "feed_forward.w3": ColwiseParallel(),
}
```

PartIR API

```
BP = ManualPartition({"x":0}, axis="B")  
AutoMP = AutomaticPartition(axis="M")  
part_fn, _ = PartIR.jit(fn, mesh, [BP, AutoMP])
```

PartIR API

```
# 1. Arrange devices in a BxM mesh.  
mesh = maps.mesh(device_array, ("B", "M"))  
# 2. Define sharding strategies as series of tactics.  
BP = ManualPartition({"x":0}, axis="B")  
MP = ManualPartition({"w1":1}, axis="M")  
Z3 = ManualPartition({"w1":0, "w2":1}, axis="B")  
schedule = [BP, MP, Z3]  
# 3. Partition and get distributed function & metadata.  
dist_fn, metadata = PartIR.jit(f, mesh, schedule)
```

Listing 5. Partitioning strategies as a series of tactics.



What's happening under the hood?

Example

$$x_2 = (xW_1)W_2$$

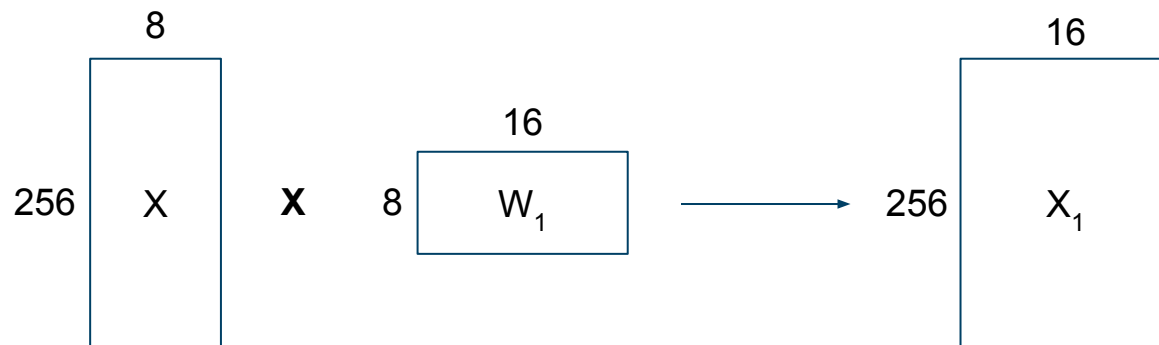
$$x_{1t} = \text{Slice}(xW_1)$$

$$x_2 = x_{1t}W_2$$

```
func @main(%x: tensor<256x8xf32>,
          %w1: tensor<8x16xf32>,
          %w2: tensor<16x8xf32>) {
    %x1 = matmul(%x, %w1)
    // value tiling
    %x1t = loop "B" [#tile<0>] (%rB: range<4>) {
        yield (slice 0 %x1[%rB])
    }
    %x2 = matmul(%x1t, %w2)
    return %x2 : tensor<256x8xf32>
```

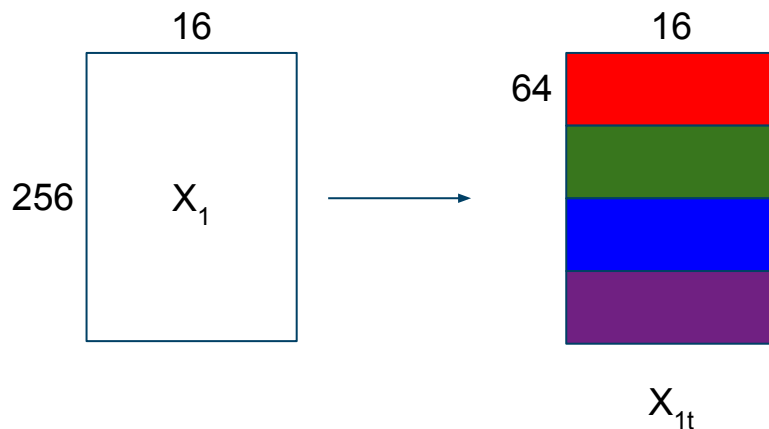
Example

`%x1 = matmul(%x, %w1)`



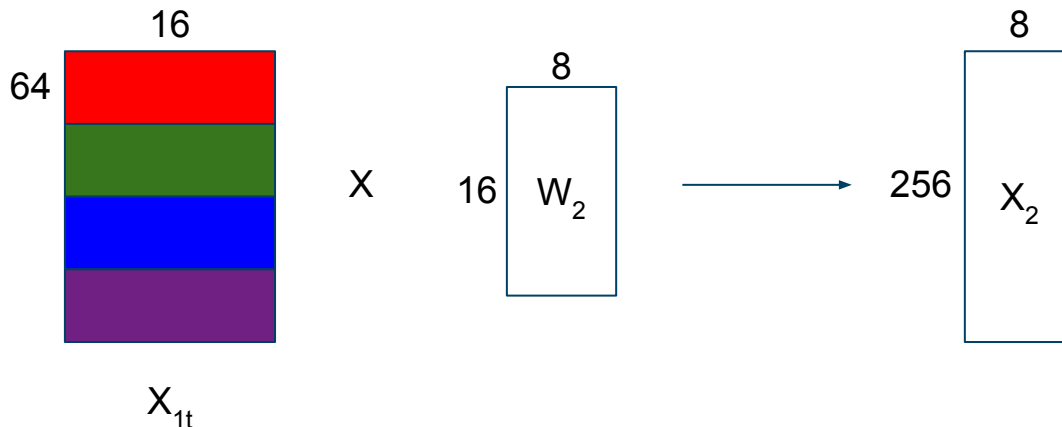
Example

```
%x1t = loop "B" [#tile<0>] (%rB: range<4>) {  
    yield (slice 0 %x1[%rB])  
}
```



Example

`%x2 = matmul(%x1t, %w2)`





Propagation Passes



```
func @main(%x: tensor<256x8xf32>,  
          %w1: tensor<8x16xf32>,  
          %w2: tensor<16x8xf32>) {  
  %x1 = matmul(%x, %w1)  
  // value tiling  
  %x1t = loop "B" [#tile<0>] (%rB: range<4>) {  
    yield (slice 0 %x1[%rB])  
  }  
  %x2 = matmul(%x1t, %w2)  
  return %x2 : tensor<256x8xf32>
```

Propagation Passes

- Value tiling on its own isn't that interesting
- Tile Mapping Registry (TMR)
 - Specifications to determine how tensor operations can be rewritten
 - Optional Tiling Actions $\rightarrow$ Arbitrary Actions

$$\begin{aligned} \text{TMR}(\text{matmul}) &= \{(\# \text{tile}\langle 0 \rangle, \perp) \hookrightarrow \# \text{tile}\langle 0 \rangle\} \\ &\cup \{(\perp, \# \text{tile}\langle 1 \rangle) \hookrightarrow \# \text{tile}\langle 1 \rangle\} \\ &\cup \{(\# \text{tile}\langle 1 \rangle, \# \text{tile}\langle 0 \rangle) \hookrightarrow \# \text{sum}\} \\ \text{TMR}(\text{add}) &= \{(\# \text{tile}\langle d \rangle, \# \text{tile}\langle d \rangle) \hookrightarrow \# \text{tile}\langle d \rangle\} \end{aligned}$$

Propagating Passes

- Propagation that isn't related to neural network training
- Forward and Backward Propagation

Propagation Passes

- Forward Propagation
 - Are there operations that use results from slicing?
- Backward Propagation
 - Are there operations that match the slice?

```
func @main(%x: tensor<256x8xf32>,  
          %w1: tensor<8x16xf32>,  
          %w2: tensor<16x8xf32>) {  
    %x1 = matmul(%x, %w1)  
    // value tiling  
    %x1t = loop "B" [#tile<0>] (%rB: range<4>) {  
        yield (slice 0 %x1[%rB])  
    }  
    %x2 = matmul(%x1t, %w2)  
    return %x2 : tensor<256x8xf32>
```



Propagation Passes

```
func @main(%x: tensor<256x8xf32>,  
          %w1: tensor<8x16xf32>,  
          %w2: tensor<16x8xf32>) {  
  %x1 = matmul(%x, %w1)  
  // value tiling  
  %x1t = loop "B" [#tile<0>] (%rB: range<4>) {  
    yield (slice 0 %x1[%rB])  
  }  
  %x2 = matmul(%x1t, %w2)  
  return %x2 : tensor<256x8xf32>
```

```
func @main(%x: ..., %w1: ..., %w2: ...) {  
  // result of backward propagation  
  %x1 = loop "B" [#tile<0>] (%rB: range<4>) {  
    yield (matmul(slice 0 %x[%rB], %w1))  
  }  
  // value tiling  
  %x1t = loop "B" [#tile<0>] (%rB: range<4>) {  
    yield (slice 0 %x1[%rB])  
  }  
  // result of forward propagation  
  %x2 = loop "B" [#tile<0>] (%rB: range<4>) {  
    yield (matmul(slice 0 %x1t[%rB], %w2))  
  }  
  return %x2 : tensor<256x8xf32>  
}
```



Propagation Passes

```
func @main(%x: ..., %w1: ..., %w2: ...) {  
    // result of backward propagation  
    %x1 = loop "B" [#tile<0>] (%rB: range<4>) {  
        yield (matmul(slice 0 %x[%rB], %w1))  
    }  
    // value tiling  
    %x1t = loop "B" [#tile<0>] (%rB: range<4>) {  
        yield (slice 0 %x1[%rB])  
    }  
    // result of forward propagation  
    %x2 = loop "B" [#tile<0>] (%rB: range<4>) {  
        yield (matmul(slice 0 %x1t[%rB], %w2))  
    }  
    return %x2 : tensor<256x8xf32>  
}
```

}

```
func @main(%x: ..., %w1: ..., %w2: ...) {  
    %r = loop "B" [#tile<0>] (%rB : range<4>) {  
        %xs = slice 0 %x[%rB] : tensor<64x8xf32>  
        %x1s = matmul(%xs, %w1)  
        %x2s = matmul(%x1s, %w2)  
    }
```

Evaluation

- Do partitioned programs achieve SOTA performance
- Automated Vs Manual Scheduling
- Sharding Decisions

Evaluation: SOTA Performance

Mesh	Size	MFU (%)		HBM (GB)	
		PartIR	GSPMD	PartIR	GSPMD
16x2 TPU	5B	58.5	58.3	14.38	14.38
32x4 TPU	32B	52.3	52.2	14.48	14.48
8x2 GPU	5B	42.2	42.9	27.02	26.73

Table 2. The MFU (higher is better) and HBM usage (lower is better) on GPUs and TPUs using PartIR and GSPMD.

Evaluation: Auto Vs Manual

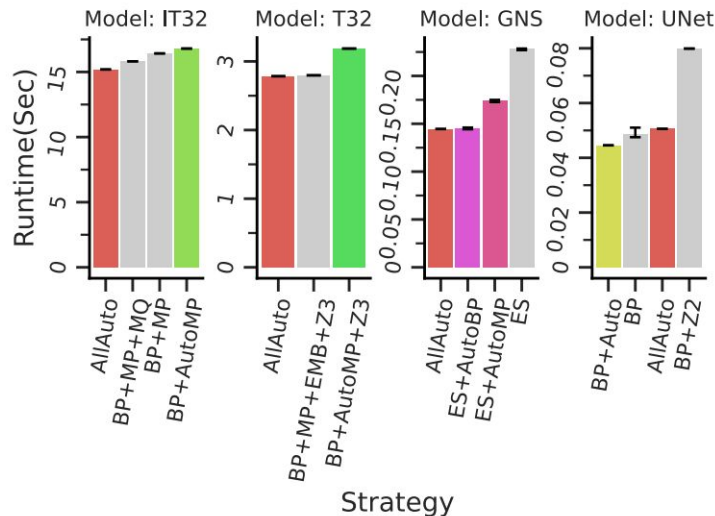


Figure 6. One-step time (lower is better) on 8x4 TPUs. Grey-colored bars indicates schedule of manual tactics, while color-coded bars are schedules including automatic tactics.

Evaluation: Sharding

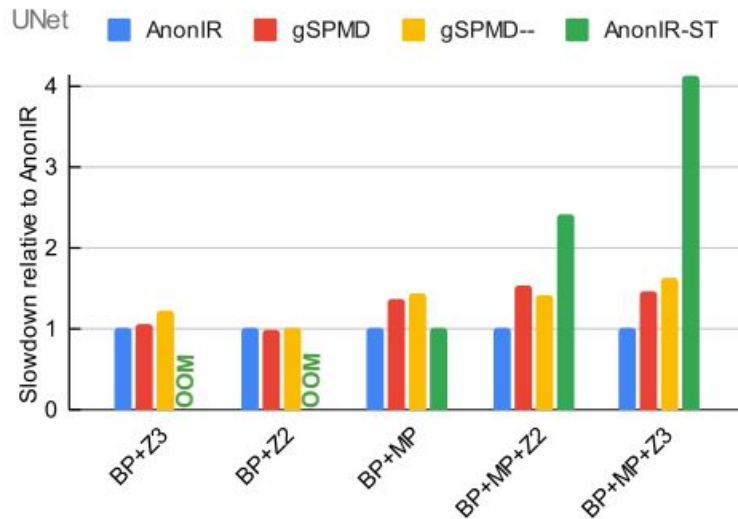


Figure 7. Relative slowdown compared to PartIR (higher is worse) for UNet partitioned on a {8:batch, 2:model} TPU.

Recap

- Aims to separate model implementation from computation scheduling
- Abstract and automate scheduling and sharding
- Used propagation strategies to partition models

Discussion

Pros

- Segments partitioning from model logic
- Easy for beginners while still allowing experts to 'tune' partitioning schedules

Cons

- Does not support MPMD

Coming Back To The Start



Comparison

	Alpa (Compiler-driven)	PartIR (IR-driven)
Goal	Automate parallelization for LM training	Make LM training predictable, and expressive
Approach	Two-level search: DP (inter-op) + ILP (intra-op)	Algebraic IR rewrites with tactics (schedule API)
User Experience	Black-box automation → @alpa.parallelize	Transparent & composable → more control, easier debugging
Focus	Performance & efficiency	Portability, maintainability, correctness
Outlook	“Let compiler decide”	“Let user compose and reason with tactics”